

Threat Intelligence IOC Enrichment Tool

Detailed Blue Team Project Documentation

Project Type	Python-based threat intelligence and IOC enrichment utility
Focus Area	Threat intelligence, IOC analysis, malware reputation, SOC automation
Primary API	VirusTotal API v3
Supported IOCs	IP addresses, domains, URLs, MD5, SHA1, and SHA256 file hashes
Outputs	Terminal report, JSON report, CSV report
Validation Evidence	Tested with a known malware test hash returning 66 malicious detections

Overview

This project is a practical blue team utility that automates the enrichment of Indicators of Compromise (IOCs) using VirusTotal. It was built to reflect a common SOC workflow: taking a suspicious IP, domain, URL, or file hash, enriching it with external threat intelligence, assigning an analyst-friendly severity level, and producing structured reports that can support triage, escalation, or incident response.

Project Objectives

- Automate enrichment for common IOC types used in SOC investigations.
- Classify IOCs into readable severity levels using malicious and suspicious verdict counts.
- Support both single IOC analysis and bulk IOC processing from a text file.
- Generate JSON and CSV reports for documentation and repeatable analysis.
- Provide clear recommendations that match typical triage and incident response workflows.

Technical Design

The tool follows a simple enrichment pipeline. The analyst provides an IOC, the script identifies the IOC type, builds the correct VirusTotal API endpoint, retrieves reputation and verdict data, applies severity logic, and saves the result in structured formats.

Step	Process	Description
1	Input	Accepts a single IOC or a text file containing multiple IOCs.
2	IOC Detection	Uses pattern matching to classify IPs, domains, URLs, MD5, SHA1, and SHA256 hashes.
3	API Enrichment	Queries the correct VirusTotal API v3 endpoint for the detected IOC type.
4	Result Parsing	Extracts malicious, suspicious, harmless, undetected, and reputation values.
5	Threat Scoring	Assigns HIGH, MEDIUM, LOW, INFO, or CLEAN/UNKNOWN severity.
6	Reporting	Prints a readable terminal report and saves JSON/CSV outputs.

Supported IOC Types

IOC Type	Example	How the Tool Handles It
IP Address	8.8.8.8	Uses the VirusTotal IP address endpoint.
Domain	example.com	Uses the VirusTotal domain endpoint.
URL	https://example.com	Encodes the URL using URL-safe Base64 and queries the URL endpoint.
File Hash	MD5 / SHA1 / SHA256	Uses the VirusTotal file endpoint for hash reputation lookup.

Threat Classification Logic

The severity model is intentionally simple and explainable. It uses vendor verdict counts returned by VirusTotal and converts them into an analyst-friendly recommendation. This makes the output easy to justify during triage or documentation.

Threat Level	Logic	Recommended Analyst Action
HIGH	20 or more malicious detections	Block the IOC and investigate related endpoint, network, and user activity.
MEDIUM	5 or more malicious or suspicious detections	Investigate the IOC and review associated SIEM, endpoint, and network logs.
LOW	1-4 malicious or suspicious detections	Review manually before taking action and validate business context.
INFO	No detections but negative reputation	Treat as suspicious context and continue validation.
CLEAN / UNKNOWN	No significant detections	No immediate malicious indicators; continue monitoring if context warrants.

Implementation Details

The project was implemented as a Python command-line tool. It uses environment variables to protect the VirusTotal API key, avoiding hard-coded credentials in the repository. The script uses request timeouts, basic API error handling, and separate functions for IOC detection, API endpoint building, verdict assignment, report generation, and file output.

- **Credential handling:** reads the API key from VT_API_KEY rather than storing it in source code.
- **IOC detection:** uses regular expressions and string checks to classify IOC types before lookup.
- **URL support:** uses URL-safe Base64 encoding, which is required for VirusTotal URL lookups.
- **Bulk processing:** reads one IOC per line from a text file and skips empty/commented lines.
- **Structured output:** saves the results to JSON and CSV files for later review.
- **Analyst output:** prints a readable terminal report with severity and recommendation fields.

Command Examples

Single IOC analysis:

```
python threat_intel_enricher.py -i 8.8.8.8
```

Known malware test hash analysis:

```
python threat_intel_enricher.py -i 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
```

Bulk IOC analysis:

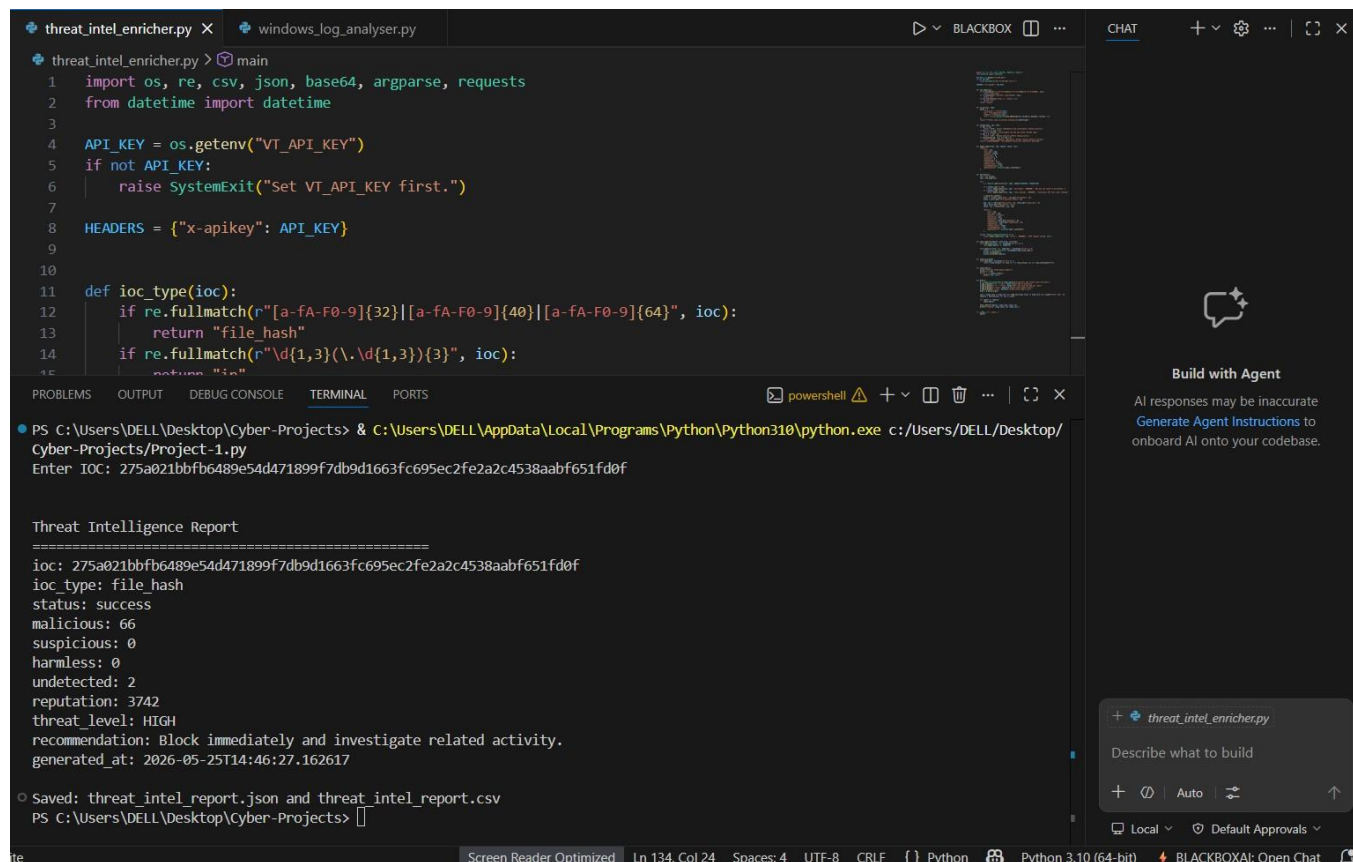
```
python threat_intel_enricher.py -f iocs.txt
```

Expected Output Fields

Field	Purpose
ioc	The analysed indicator value.
ioc_type	Detected IOC category: IP, domain, URL, or file hash.
status	Shows whether the lookup succeeded, failed, was rate-limited, or was not found.
malicious	Number of VirusTotal vendors that marked the IOC as malicious.
suspicious	Number of vendors that marked the IOC as suspicious.
harmless	Number of vendors that marked the IOC as harmless.
undetected	Number of vendors that did not detect the IOC.
reputation	VirusTotal reputation score where available.
threat_level	Calculated severity level produced by the tool.
recommendation	SOC-style action guidance based on the severity level.
generated_at	Timestamp of the enrichment result.

Validation Evidence

The tool was tested in Visual Studio Code using a known malware test hash. The execution result confirms that the script correctly detected the IOC as a file hash, queried VirusTotal successfully, returned 66 malicious detections, assigned a HIGH threat level, and generated both JSON and CSV report files.



```
threat_intel_enricher.py x windows_log_analyser.py
threat_intel_enricher.py > main
1 import os, re, csv, json, base64, argparse, requests
2 from datetime import datetime
3
4 API_KEY = os.getenv("VT_API_KEY")
5 if not API_KEY:
6     raise SystemExit("Set VT_API_KEY first.")
7
8 HEADERS = {"x-apikey": API_KEY}
9
10
11 def ioc_type(ioc):
12     if re.fullmatch(r"[a-fA-F0-9]{32}||[a-fA-F0-9]{40}||[a-fA-F0-9]{64}", ioc):
13         return "file_hash"
14     if re.fullmatch(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}", ioc):
15         return "ip"
16
17 PS C:\Users\DELL\Desktop\Cyber-Projects> & C:\Users\DELL\AppData\Local\Programs\Python\Python310\python.exe c:/Users/DELL/Desktop/Cyber-Projects/Project-1.py
Enter IOC: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabb651fd0f

Threat Intelligence Report
=====
ioc: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabb651fd0f
ioc_type: file_hash
status: success
malicious: 66
suspicious: 0
harmless: 0
undetected: 2
reputation: 3742
threat_level: HIGH
recommendation: Block immediately and investigate related activity.
generated_at: 2026-05-25T14:46:27.162617

Saved: threat_intel_report.json and threat_intel_report.csv
PS C:\Users\DELL\Desktop\Cyber-Projects>
```

Figure 1: Successful execution of the IOC enrichment tool showing a HIGH severity result and generated JSON/CSV reports.

Validated Test Result

Test IOC	275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabb651fd0f
Detected Type	file_hash
Lookup Status	success
Malicious Verdicts	66
Suspicious Verdicts	0
Threat Level	HIGH
Generated Reports	threat_intel_report.json and threat_intel_report.csv

Repository Structure

```
THREAT-INTELLIGENCE-IOC-ENRICHMENT-TOOL/  
|-- README.md  
|-- threat_intel_enricher.py  
|-- Threat_Intelligence_IOC_Enrichment_Detailed_Project_Documentation.pdf
```

Operational SOC Use Cases

- Enriching suspicious IPs, domains, URLs, or hashes identified from SIEM alerts.
- Checking malware hashes from email attachments or endpoint detections.
- Supporting triage by quickly identifying whether an IOC has high malicious vendor detections.
- Generating lightweight evidence files for incident notes or investigation handover.
- Preparing IOC lists before blocking, monitoring, or escalation decisions.

Security and Handling Considerations

- The VirusTotal API key is not stored in the code or uploaded to GitHub.
- The tool performs lookup and enrichment only; it does not execute files or malware.
- CSV and JSON reports should be reviewed before sharing externally because they may contain sensitive IOCs.
- VirusTotal community and vendor results should support analyst judgement, not replace investigation context.
- API rate limits must be considered when processing large IOC lists.

Limitations

- The tool depends on VirusTotal availability, API limits, and available reputation data.
- Vendor detections can change over time as security engines update their results.
- A clean or unknown result does not guarantee that an IOC is safe.
- The severity model is rule-based and should be adjusted for organisational risk tolerance.
- The current version does not perform sandbox detonation, WHOIS lookup, passive DNS analysis, or SIEM ingestion.

Future Enhancements

- ANY.RUN sandbox report integration for behavioural malware analysis.
- AbuseIPDB integration for additional IP reputation scoring.
- MITRE ATT&CK; mapping based on IOC context and sandbox behaviour.
- HTML or PDF report generation directly from the Python tool.
- Optional SIEM export format for Splunk, Elastic, or Microsoft Sentinel workflows.
- Screenshot or evidence capture support for stronger investigation documentation.

Conclusion

This project demonstrates a practical and defensible blue team workflow using Python, API integration, IOC classification, threat scoring, and structured reporting. The validation result confirms that the tool can successfully enrich a known malicious file hash and convert the returned intelligence into an analyst-friendly severity and recommendation. It is suitable as an entry-level SOC and threat intelligence portfolio project because it connects Python automation directly to real-world investigation tasks.